

ДЗ_8_АЙКОС

Комкова Полина Дмитриевна БПИ244

Формулировка ДЗ

Разработать независимые программы **клиента и сервера**, взаимодействующие через **разделяемую память** с использованием функций POSIX.

Клиент в автоматическом режиме **генерирует случайные числа** в том же диапазоне, что и клиент, рассмотренный на семинаре (**то есть от 0 до 999**).

Сервер осуществляет **вывод данных** из разделяемой памяти. Предполагается, что запускаются только один клиент и один сервер.

Необходимо обеспечить корректное завершение работы для одного клиента и одного сервера, при котором **удаляется сегмент разделяемой памяти и завершаются оба процесса при попытке завершить один из них**. В качестве одного из вариантов решения можно использовать **сигналы**, рассмотренные на лекции

Исходный код

```
// message.h

#include <sys/types.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
#include <signal.h>

#define PERMS 0666

// Коды сообщений
#define MSG_TYPE_EMPTY 0 // Разделяемая память пуста
#define MSG_TYPE_NUMBER 1 // Передается число
#define MSG_TYPE_FINISH 2 // Сообщение о завершении
```

```

// Структура сообщения
typedef struct {
    int type;
    int number; // случайное число
    pid_t server_pid; // pid сервера для отправки сигналов
    pid_t client_pid; // pid клиента для отправки сигналов
} message_t;

const char* shar_object = "random-numbers-shm";

// Функций для работы с сигналами
void setup_signal_handlers(void);
void handle_signal(int sig);

```

```

// shared-memory-server.c

#include "message.h"

// Флаг завершения с гарантией, что каждое чтение переменной будет из памяти
volatile sig_atomic_t shutdown_flag = 0;

// Обработчик сигналов
void handle_signal(int signal) {
    printf("\nСервер: получен сигнал %d, завершаем работу...\n", signal);
    shutdown_flag = 1;
}

// Настройка обработчиков сигналов
void setup_signal_handlers(void) {
    signal(SIGINT, handle_signal); // Ctrl+C в термнале
    signal(SIGTERM, handle_signal); // Команда kill в коде
}

void sys_err (char *msg) {
    puts (msg);
    exit (1);
}

int main () {
    int shmid;
    message_t *msg_p;

    // Настройка обработчиков сигналов
    setup_signal_handlers();

```

```

// Создаем разделяемую память
if ( (shmid = shm_open(shar_object, O_CREAT|O_RDWR, 0666)) == -1 ) {
    perror("shm_open");
    sys_err ("Сервер: объект уже открыт");
} else {
    printf("Сервер: объект открыт: имя = %s, id = 0x%x\n", shar_object,
shmid);
}
// Задаем размер объекта памяти
if (ftruncate(shmid, sizeof (message_t)) == -1 ) {
    perror("ftruncate");
    sys_err ("Сервер: ошибка определения размера памяти");
    return 1;
} else {
    printf("Сервер: объем памяти установлен и равен %lu\n", sizeof
(message_t));
}

// Получаем доступ к памяти
msg_p = mmap(0, sizeof (message_t), PROT_WRITE|PROT_READ, MAP_SHARED, shmid,
0);
if (msg_p == (message_t*)-1 ) {
    perror("mmap");
    sys_err ("Сервер: неправильный доступ к памяти");
}

// Инициализируем
msg_p->type = MSG_TYPE_EMPTY;
msg_p->number = 0;
msg_p->server_pid = getpid(); // Сохраняем pid сервера
msg_p->client_pid = 0; // Пока клиент не подключился
printf("Сервер запущен (pid: %d). Ждем ввод чисел...\n", getpid());
printf("\nНажмите Ctrl+C, чтобы остановить работу сервера и клиента\n");

int numbers_received = 0; // Обработанные числа
while (shutdown_flag == 0) {
    // Если есть сообщение
    if (msg_p->type != MSG_TYPE_EMPTY) {
        // Если это число
        if (msg_p->type == MSG_TYPE_NUMBER) {
            numbers_received++;
            printf("Сервер получил число %d: %d\n", numbers_received, msg_p-
>number);
        } else if (msg_p->type == MSG_TYPE_FINISH) { // Иначе если финиш
            printf("Сервер: получен сигнал завершения от клиента\n");

```

```

        break;
    }
    msg_p->type = MSG_TYPE_EMPTY; // Сообщение обработано
}
usleep(100000); // Остановка на 100ms, чтобы цикл с сумасшедшей скоростью
не проходил миллионы раз за просто так
}

// Если сервер завершается по сигналу, уведомляем клиента
if (shutdown_flag && msg_p->client_pid != 0) {
    printf("Сервер: уведомляет клиента (pid: %d) о завершении...\n", msg_p-
>client_pid);
    kill(msg_p->client_pid, SIGTERM); // Отправляем сигнал клиенту
    sleep(1); // Даем время клиенту на обработку
}

// Удаление разделяемой памяти
printf("Сервер: удаление общей памяти...\n");
if(shm_unlink(shar_object) == -1) {
    perror("shm_unlink");
    sys_err ("Сервер: ошибка при удалении общей памяти");
} else {
    printf("Сервер: общая память успешно удалена\n");
}

printf("Сервер: отключен. Общее количество полученных сообщений: %d\n",
numbers_received);
return 0;
}

```

```

// shared-memory-client.c

#include "message.h"

// Флаг завершения с гарантией, что каждое чтение переменной будет из памяти
volatile sig_atomic_t shutdown_flag = 0;

// Обработчик сигналов
void handle_signal(int signal) {
    printf("\nКлиент: получен сигнал %d, инициирую завершение работы...\n",
signal);
    shutdown_flag = 1;
}

// Настройка обработчиков сигналов

```

```

void setup_signal_handlers(void) {
    signal(SIGINT, handle_signal);    // Ctrl+C в термнале
    signal(SIGTERM, handle_signal);  // Команда kill в коде
}

void sys_err (char *msg) {
    puts (msg);
    exit (1);
}

int main () {
    int shmid;
    message_t *msg_p;

    // Настройка обработчиков сигналов
    setup_signal_handlers();
    // Создаем разделяемую память
    if ( (shmid = shm_open(shar_object, O_CREAT|O_RDWR, 0666)) == -1 ) {
        perror("shm_open");
        sys_err ("Клиент: объект уже открыт");
    } else {
        printf("Клиент: объект открыт: имя = %s, id = 0x%x\n", shar_object,
shmid);
    }

    // Получаем доступ к памяти
    msg_p = mmap(0, sizeof (message_t), PROT_WRITE|PROT_READ, MAP_SHARED, shmid,
0);
    if (msg_p == (message_t*)-1 ) {
        perror("mmap");
        sys_err ("Клиент: неправильный доступ к памяти");
    }

    // Сохраняем pid клиента в разделяемую память
    msg_p->client_pid = getpid();
    printf("Клиент запущен (pid: %d). Генерируем случайные числа...\n",
getpid());
    printf("\nНажмите Ctrl+C, чтобы остановить работу клиента и сервера\n");

    srand(time(NULL)); // Инициализация генератора случайных чисел
    int numbers_sent = 0; // Отправленные числа
    while (shutdown_flag == 0) {
        // Пока сервер еще не обработал предыдущее число и не получен сигнал
        while (msg_p->type != MSG_TYPE_EMPTY && !shutdown_flag) {
            usleep(100000); // Ждем 100ms до следующего чтения
        }
    }
}

```

```

// Если клиент завершается по сигналу
if (shutdown_flag) {
    break;
}
int random_num = rand() % 1000; // Число от 0 до 999
numbers_sent++;
// Отправляем число
msg_p->type = MSG_TYPE_NUMBER;
msg_p->number = random_num;
printf("Клиент отправил число %d: %d\n", numbers_sent, random_num);
usleep(500000); // 500ms между числами
}

// Если клиент завершается по сигналу, сообщаем серверу
if (shutdown_flag && msg_p->server_pid != 0) {
    printf("Клиент: уведомляет сервер (pid: %d) о завершении...\n", msg_p->server_pid);
    kill(msg_p->server_pid, SIGTERM); // Отправляем серверу сигнал
    // Также отправляем сообщение о завершении через разделяемую память
    while (msg_p->type != MSG_TYPE_EMPTY) {
        usleep(100000);
    }
    msg_p->type = MSG_TYPE_FINISH;
}

close(shmid);
printf("Клиент: вышел. Общее количество отправленных сообщений: %d\n",
numbers_sent);
return 0;
}

```

Идея

- **Клиент** автоматически генерирует случайные числа
- **Сервер** принимает и отображает эти числа
- Оба процесса могут корректно завершаться по сигналу `Ctrl+C`
- Используется механизм **сигналов**

Алгоритм работы

Сервер (создатель памяти):

1. `shm_open()` - создает/открывает сегмент с `O_CREAT | O_RDWR`
2. `ftruncate()` - задает размер сегмента под структуру `message_t`
3. `mmap()` - отображает память в адресное пространство процесса
4. **Работа с данными** - читает/пишет в отображенную область
5. `shm_unlink()` - удаляет сегмент при завершении

Клиент (пользователь памяти):

1. `shm_open()` - открывает существующий сегмент с `O_RDWR`
2. `mmap()` - отображает память в адресное пространство процесса
3. **Работа с данными** - читает/пишет в отображенную область
4. `close()` - закрывает дескриптор (не удаляет сегмент)

Пояснения

- **Разделяемая память POSIX**

- **Имя сегмента:** "random-numbers-shm"
- **Структура данных:**

```
typedef struct {  
    int type;           // тип сообщения  
    int number;         // случайное число  
    pid_t server_pid;   // pid сервера  
    pid_t client_pid;   // pid клиента  
} message_t;
```

- **Синхронизация**

Клиент:

1. Ждет `MSG_TYPE_EMPTY` (то есть сервер обработал предыдущее число)
2. Записывает число и устанавливает `MSG_TYPE_NUMBER`
3. Ждет 500ms

Сервер:

4. Постоянно проверяет разделяемую память
5. При `MSG_TYPE_NUMBER` - выводит число
6. Устанавливает `MSG_TYPE_EMPTY` (сигнал клиенту)
7. Ждет 100ms между проверками

- **Сигналы**

- Вариант 1: Ctrl+C в сервере**

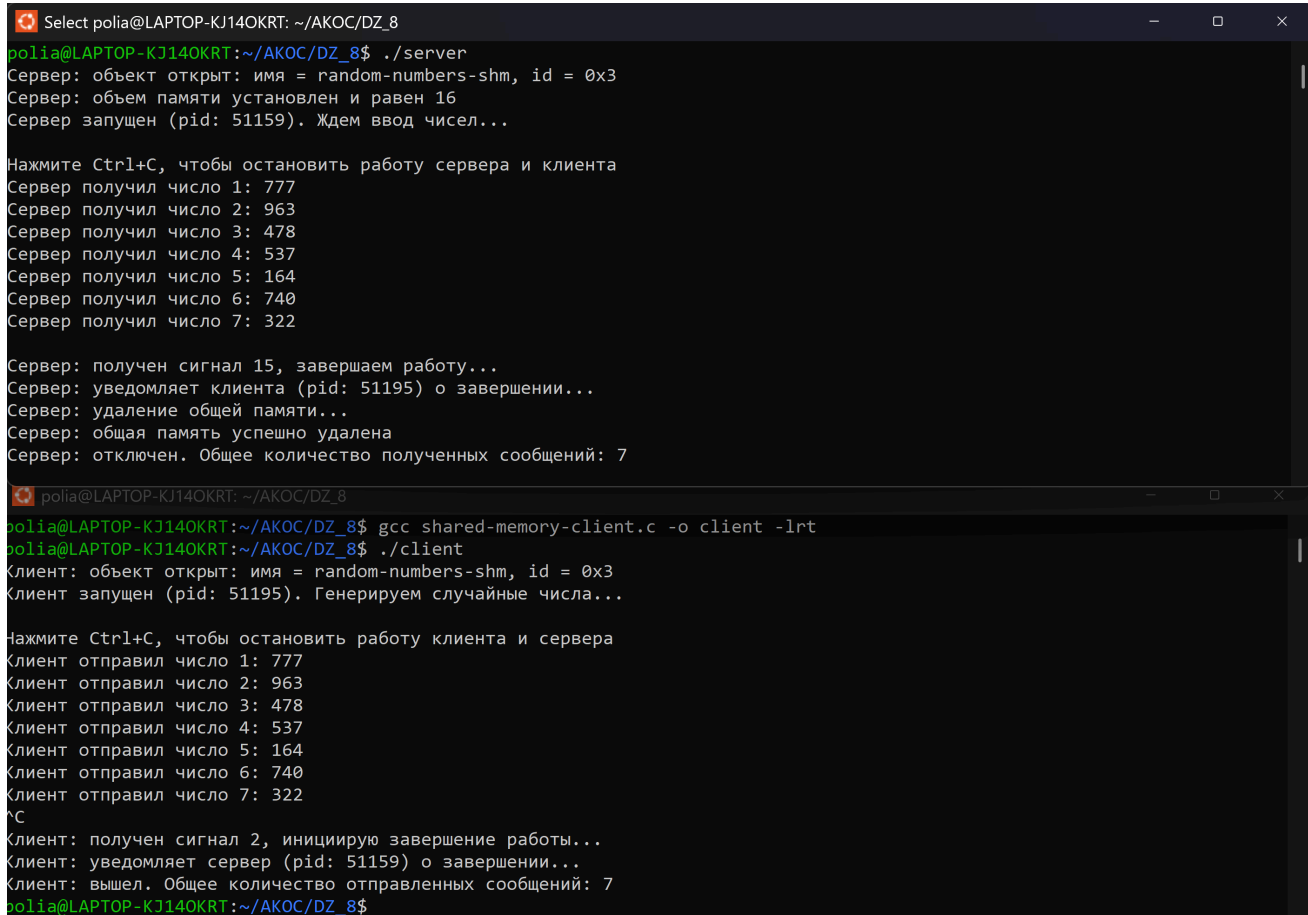
1. Сервер получает SIGINT
2. Сервер отправляет SIGTERM клиенту
3. Сервер удаляет разделяемую память
4. Клиент получает SIGTERM и корректно завершается

- Вариант 2: Ctrl+C в клиенте**

5. Клиент получает SIGINT
6. Клиент отправляет SIGTERM серверу
7. Клиент отправляет сообщение о завершении через разделяемую память
8. Сервер получает SIGTERM и корректно завершается, удаляя память

Пример использования

- **Завершение программы со стороны Клиента**



```
Select polia@LAPTOP-KJ14OKRT: ~/AKOC/DZ_8
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_8$ ./server
Сервер: объект открыт: имя = random-numbers-shm, id = 0x3
Сервер: объем памяти установлен и равен 16
Сервер запущен (pid: 51159). Ждем ввод чисел...

Нажмите Ctrl+C, чтобы остановить работу сервера и клиента
Сервер получил число 1: 777
Сервер получил число 2: 963
Сервер получил число 3: 478
Сервер получил число 4: 537
Сервер получил число 5: 164
Сервер получил число 6: 740
Сервер получил число 7: 322

Сервер: получен сигнал 15, завершаем работу...
Сервер: уведомляет клиента (pid: 51195) о завершении...
Сервер: удаление общей памяти...
Сервер: общая память успешно удалена
Сервер: отключен. Общее количество полученных сообщений: 7

polia@LAPTOP-KJ14OKRT: ~/AKOC/DZ_8
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_8$ gcc shared-memory-client.c -o client -lrt
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_8$ ./client
Клиент: объект открыт: имя = random-numbers-shm, id = 0x3
Клиент запущен (pid: 51195). Генерируем случайные числа...

Нажмите Ctrl+C, чтобы остановить работу клиента и сервера
Клиент отправил число 1: 777
Клиент отправил число 2: 963
Клиент отправил число 3: 478
Клиент отправил число 4: 537
Клиент отправил число 5: 164
Клиент отправил число 6: 740
Клиент отправил число 7: 322
^C
Клиент: получен сигнал 2, иницирую завершение работы...
Клиент: уведомляет сервер (pid: 51159) о завершении...
Клиент: вышел. Общее количество отправленных сообщений: 7
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_8$
```


- **Завершение программы со стороны Сервера**

```
polia@LAPTOP-KJ14OKRT: ~/AKOC/DZ_8
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_8$ ./server
Сервер: объект открыт: имя = random-numbers-shm, id = 0x3
Сервер: объем памяти установлен и равен 16
Сервер запущен (pid: 51681). Ждем ввод чисел...

Нажмите Ctrl+C, чтобы остановить работу сервера и клиента
Сервер получил число 1: 561
Сервер получил число 2: 68
Сервер получил число 3: 626
Сервер получил число 4: 118
^C
Сервер: получен сигнал 2, завершаем работу...
Сервер: уведомляет клиента (pid: 51694) о завершении...

Сервер: получен сигнал 15, завершаем работу...
Сервер: удаление общей памяти...
Сервер: общая память успешно удалена
Сервер: отключен. Общее количество полученных сообщений: 4
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_8$

polia@LAPTOP-KJ14OKRT: ~/AKOC/DZ_8
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_8$ ./client
Клиент: объект открыт: имя = random-numbers-shm, id = 0x3
Клиент запущен (pid: 51694). Генерируем случайные числа...

Нажмите Ctrl+C, чтобы остановить работу клиента и сервера
Клиент отправил число 1: 561
Клиент отправил число 2: 68
Клиент отправил число 3: 626
Клиент отправил число 4: 118

Клиент: получен сигнал 15, иницирую завершение работы...
Клиент: уведомляет сервер (pid: 51681) о завершении...
Клиент: вышел. Общее количество отправленных сообщений: 4
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_8$
```

Структура файла

```
HW_8/
├── message.h           # Заголовный файл объявления
├── shared-memory-client.c # Исполняемый файл клиента
├── shared-memory-server.c # Исполняемый файл сервера
└── README.md          # Описание программы и тестов
```

Компиляция

- *Запускаем 2 терминала.* Первый - для работы **сервера**

```
gcc shared-memory-server.c -o server -lrt
```

- Второй - для **клиента**

```
gcc shared-memory-client.c -o client -lrt
```

Тестирование

- Сначала запускаем **сервер**

```
./server
```

- Потом - **клиента**

```
./client
```